

Data Structure

12. Dijkstra & A*

12주차: 다익스트라, A*

목차 INDEX

01 BFS

- Graph
- BFS
- Heap
- Priority Queue
- 최단거리

IM 상급 ~ A형 기초

02 Dijkstra

- Selection
- Relaxation
- Distance Array
- 경로 재구축
- Grid Dijkstra

A형 상급 ~ B형 기초

03 A*

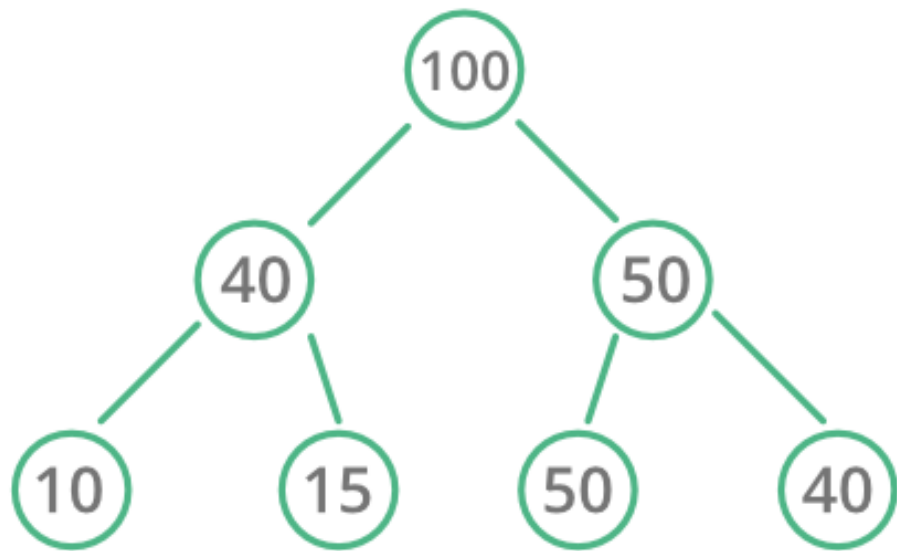
- Heuristic
- $g(n)$, $h(n)$, $f(n)$
- Manhattan Distance
- Grid A*

B형

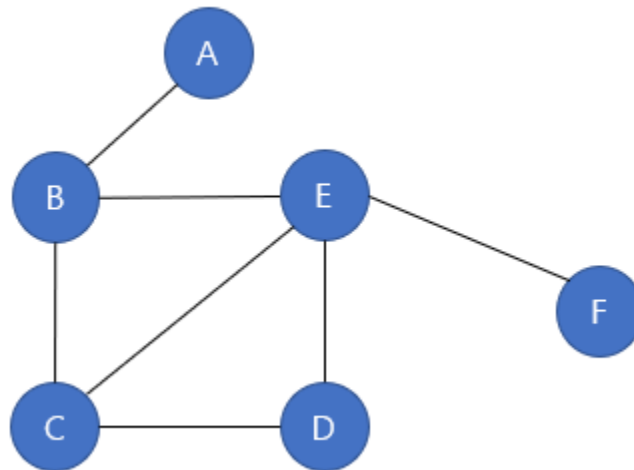
01 BFS

Breadth-First Search

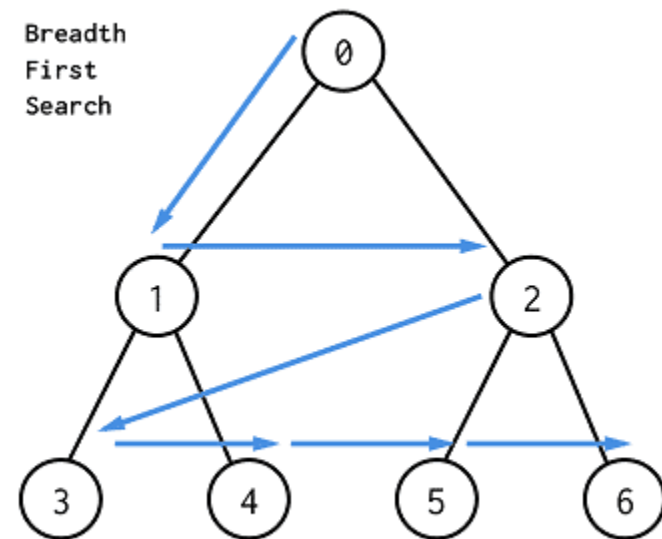
오늘의 준비물



Heap



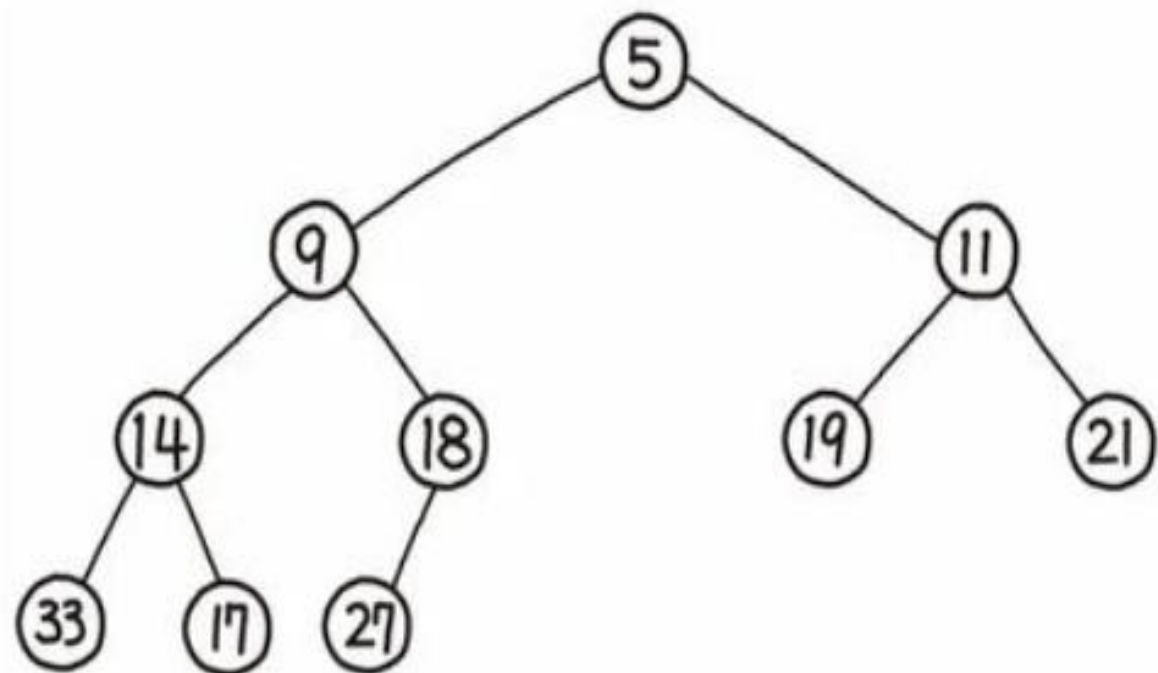
Graph



BFS

Heap Tree (or Heap)

- 완전 이진 트리의 조건을 만족해야 함
- 부모와 자식 노드 사이 크기 규칙을 만족해야 함
(이 규칙에 따라 무슨 힙인지가 갈림)
- 부모가 항상 자식보다 크면 Max Heap,
항상 작으면 Min Heap이라 부름
- 오른쪽 이미지는 Min Heap을 나타냄
- Heap의 루트 노드는 항상 최댓값 또는
최솟값이 있기 때문에 최솟값 최댓값
확인 연산(Peek)이 $O(1)$ 임



그래프

SNS

사람 = Node
친구 = Edge

지하철

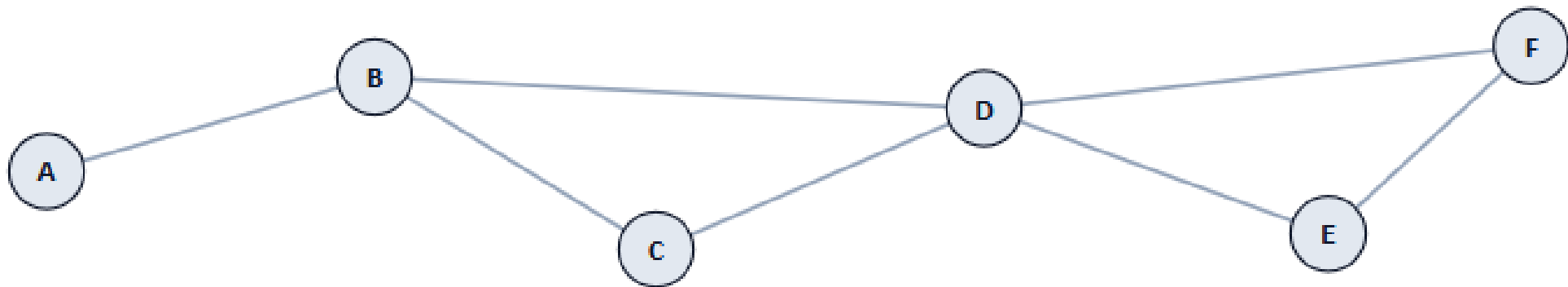
역 = Node
노선 = Edge

도로

교차로 = Node
도로 = Edge

웹

페이지 = Node
링크 = Edge



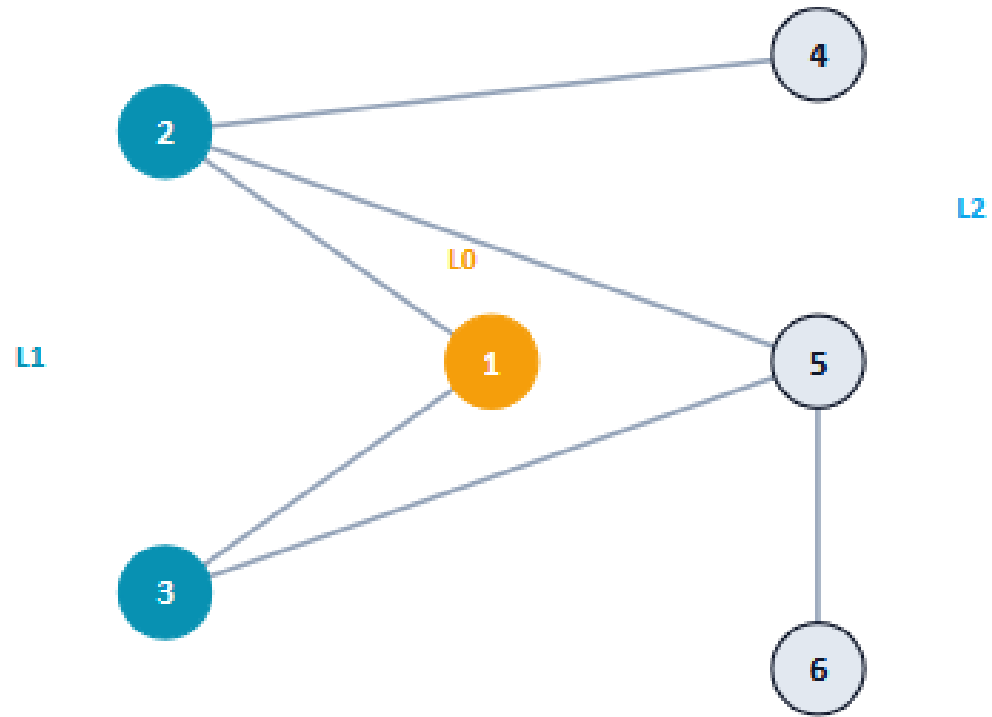
복잡한 관계를 추상화하는 기법

그래프의 용어는 트리와 공유함 (트리 $\subset$ 그래프)

BFS: Breadth-First Search

시작점에서 가까운 Node부터

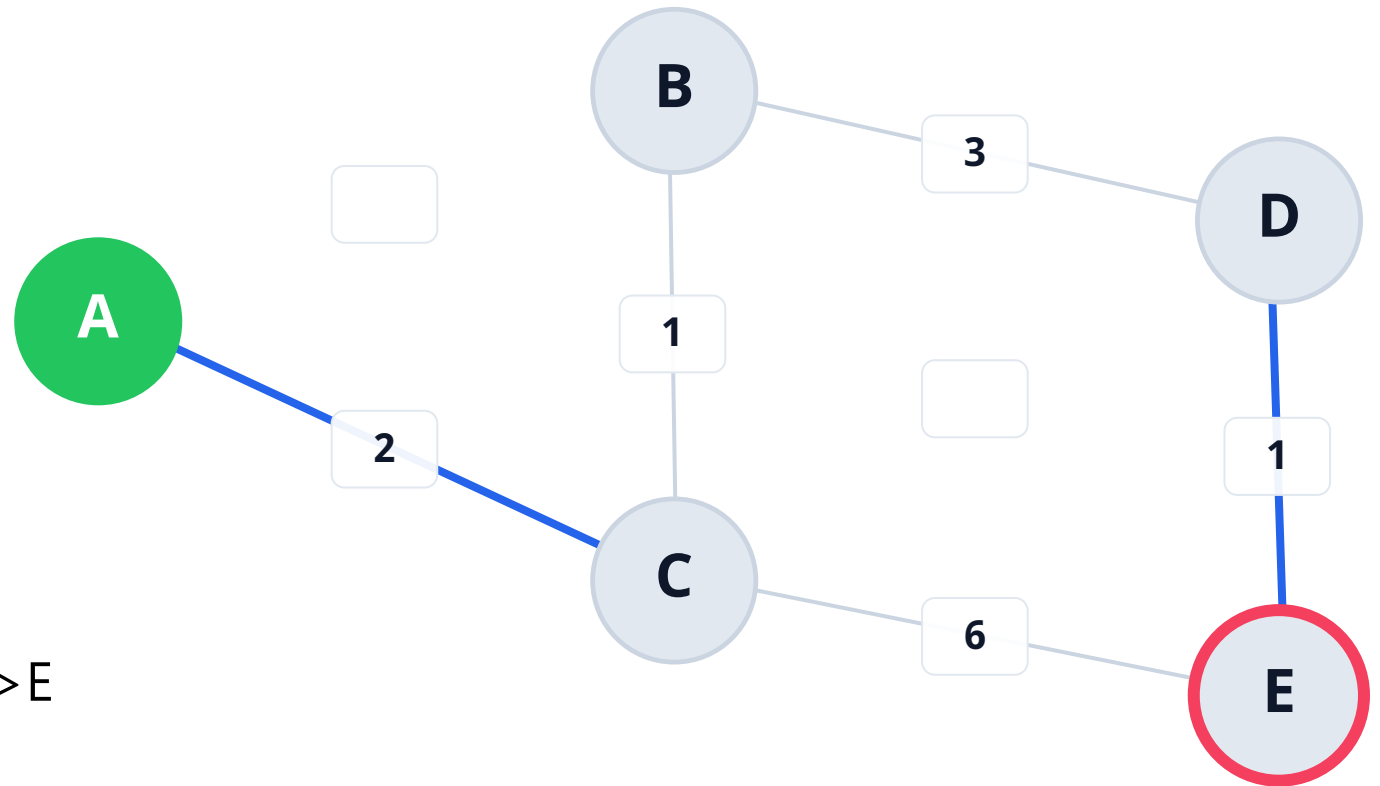
Level 0 → Level 1 → Level 2
파동처럼 바깥으로 확장



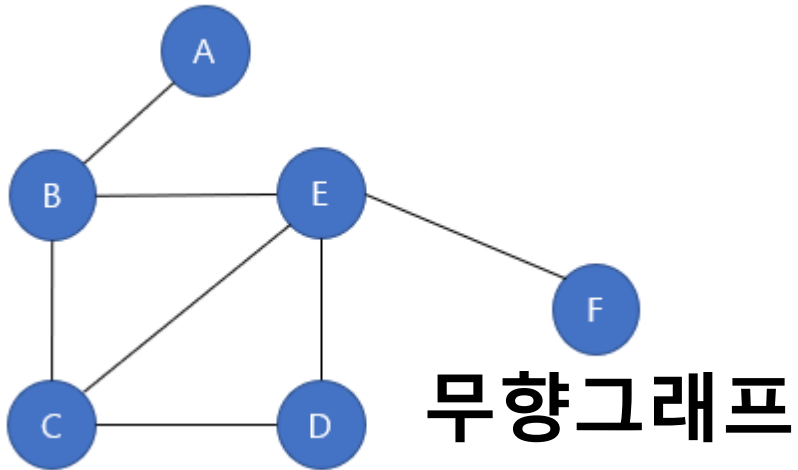
그래프에서의 최단거리

Start A → Goal E

- 비가중 Graph
→ 지나간 Edge 수가 최소: A>C>E
- 가중 Graph
→ Edge Weight의 합이 최소: A>C>B>D>E



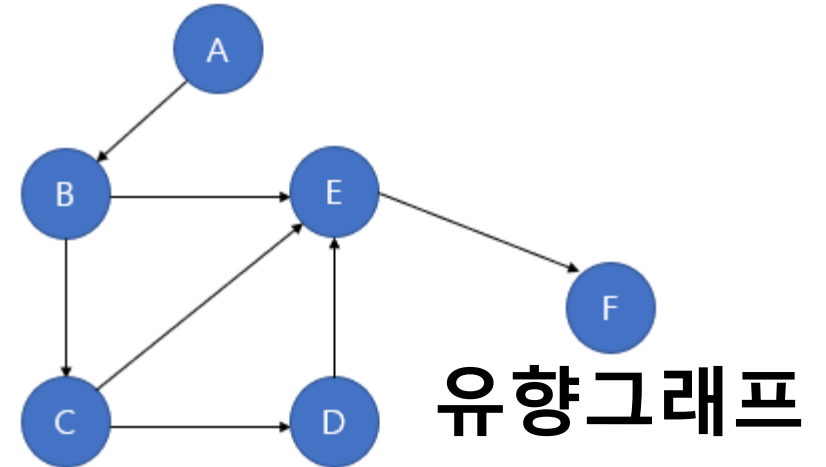
그래프에서의 최단거리



FIFO Queue가 보는 것

“먼저 발견했는가?”

➡ **Breadth-First Search**



Weighted Shortest Path가 필요한 것

“현재 누적 비용이 가장 작은가?”

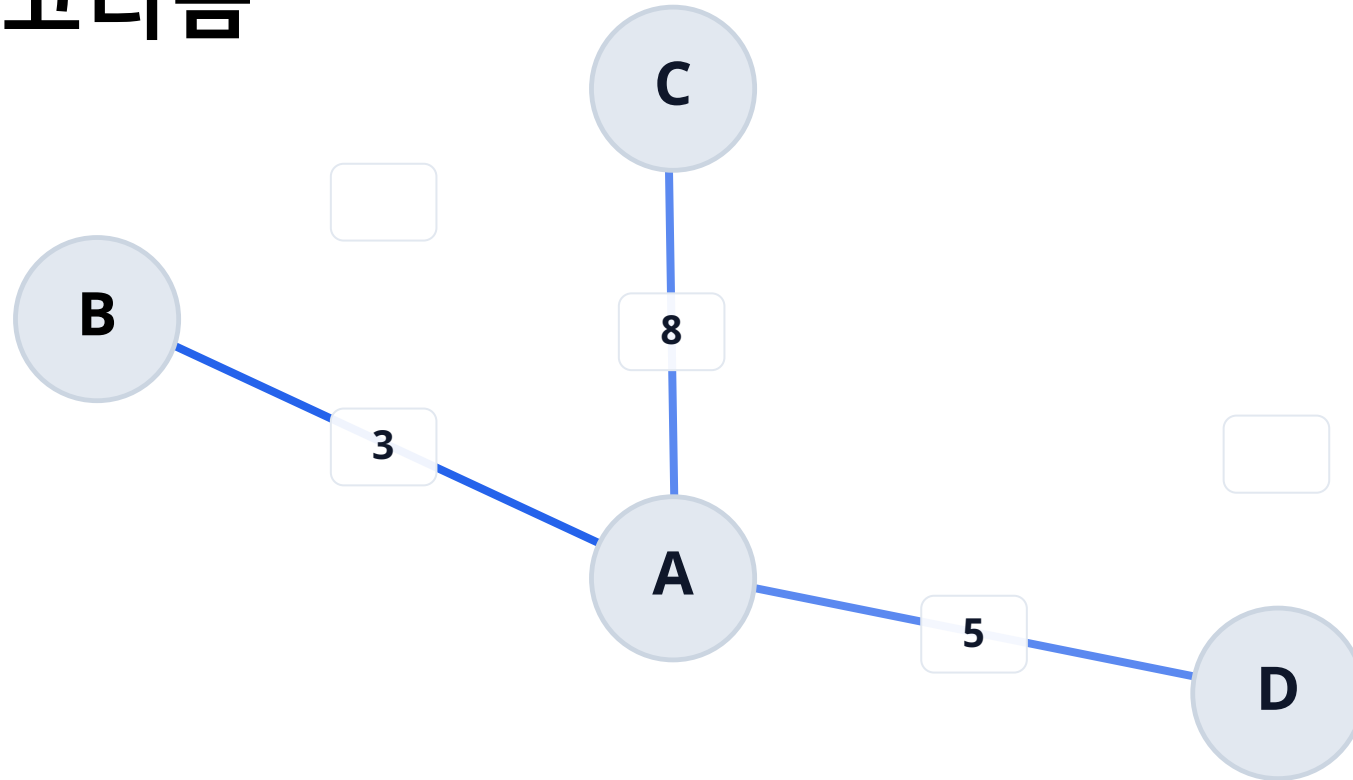
➡ **Dijkstra**

02

다익스트라

Dijkstra Algorithm

다익스트라 알고리즘



다음으로 탐색해야 할 후보 노드: B

다익스트라 알고리즘

① Closest Candidate

다음 처리할 후보 중
거리가 가장 가까운 Vertex 선택

Priority Queue로 구현

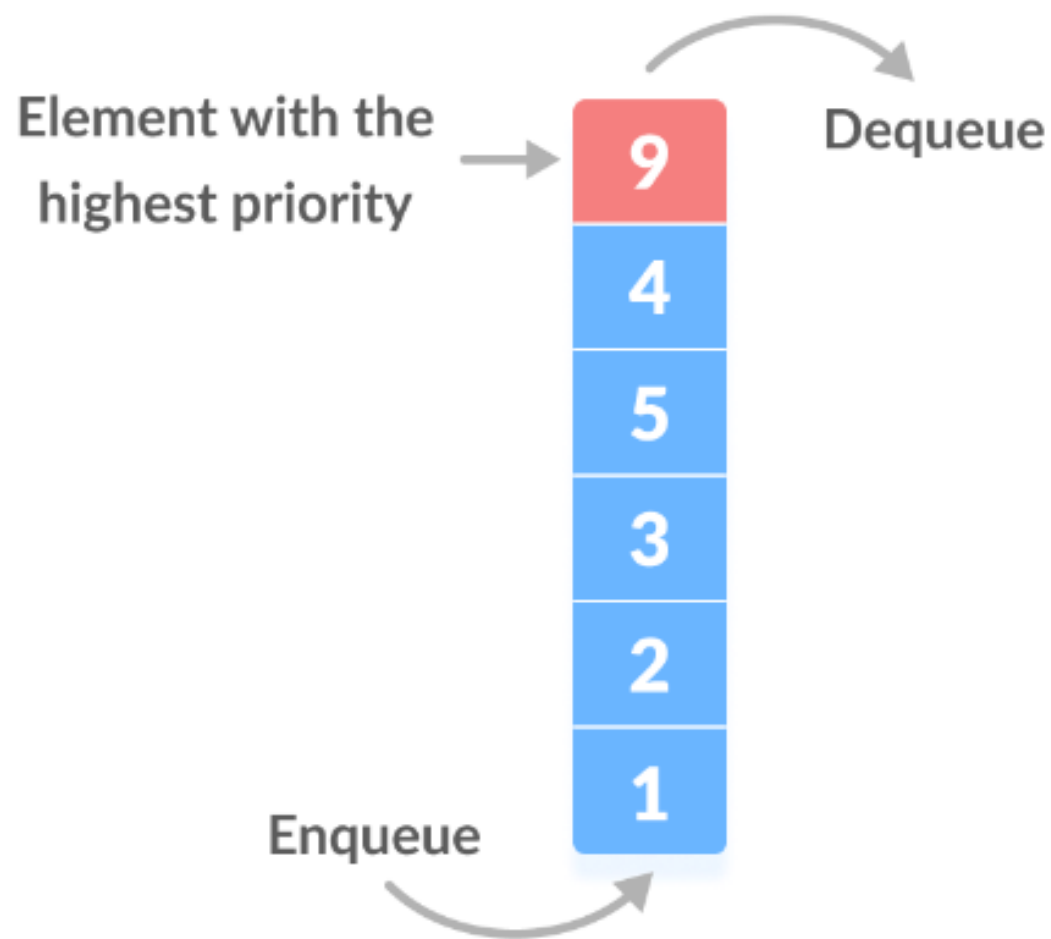
② Relaxation

그 Vertex를 거쳐 가는 것이
최종적으로 더 짧다면 거리 갱신

다익스트라 알고리즘은
이 두 단계의 반복으로 이루어짐

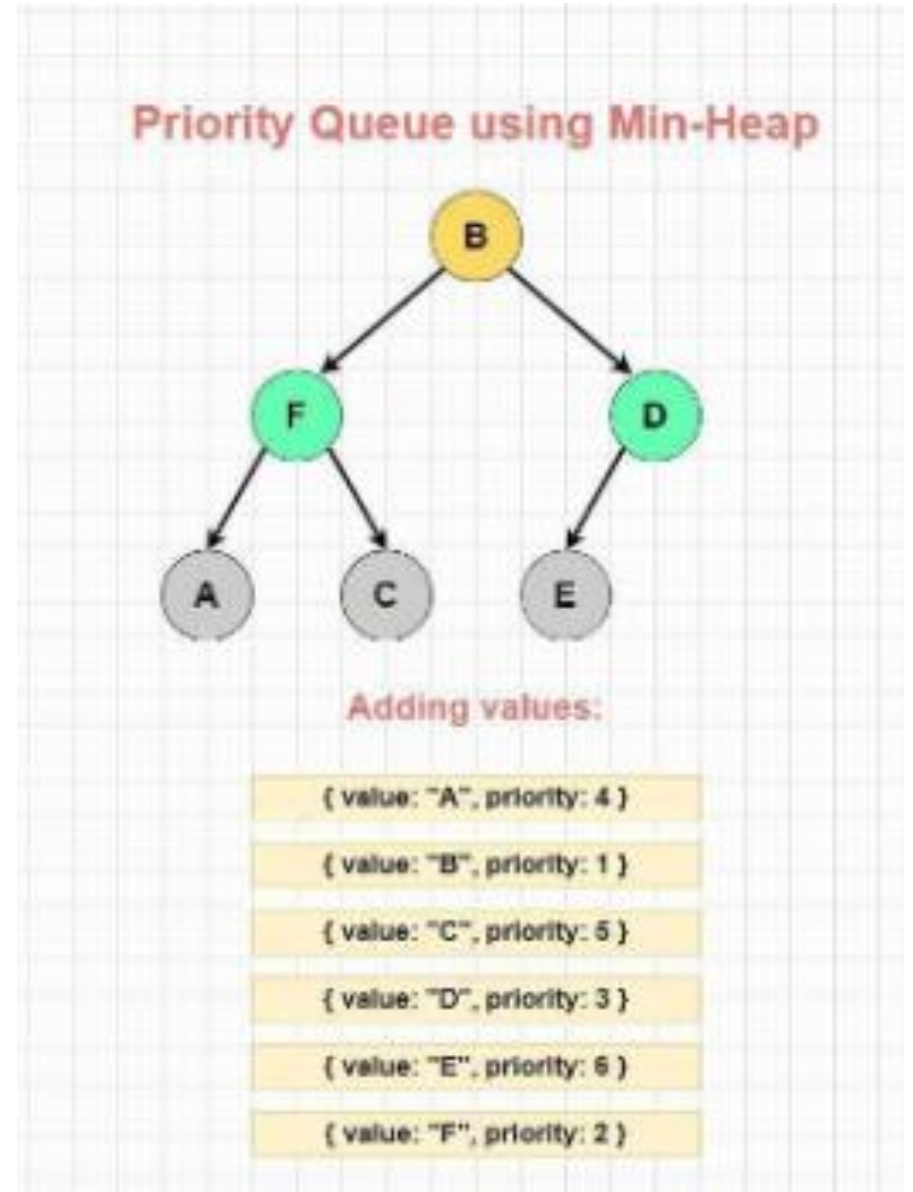
Priority Queue

- 우선순위가 높은 데이터부터 처리하는 구조
- 들어온 순서와 관계없이 우선순위에 따라 처리 순서 결정
- 데이터 삽입은 insert, 가장 우선순위가 높은 데이터 제거는 pop이라 함 (Enqueue/Dequeue 로도 표현함)
- 데이터마다 우선순위라는 Key를 부여해서 처리 시 가장 높은 Key를 우선 선택하는 방식
- OS의 프로세스 스케줄링, Dijkstra Algorithm 등에서 쓰임

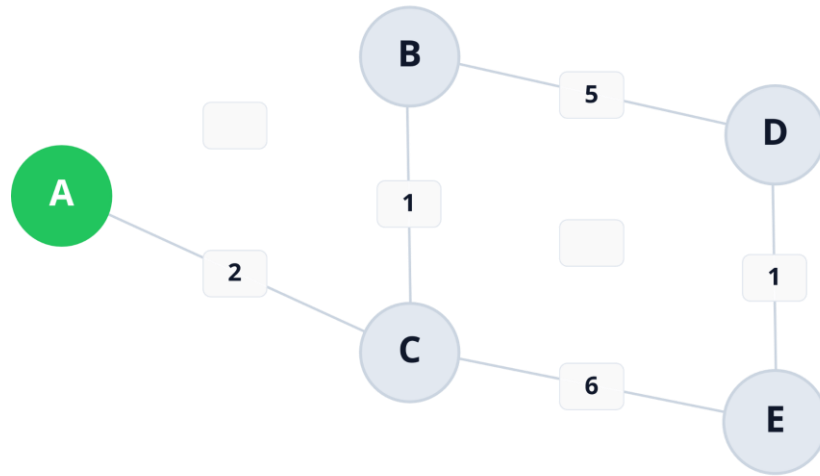


Closest Candidate Selection

- 데이터가 A, B, C, D, E, F 순으로 들어온 상황
- 해당 데이터들을 Min-Heap으로 정리
(데이터가 하나 들어올 때마다 Heapify 가 일어남)
- 우선순위가 가장 작은 B가 Root에 오게 됨
Heap(으로 구현된 우선순위 큐)에서 Pop 연산을 통해
가장 작은 우선순위부터 빼내기 가능



dist Array



Priority Queue

[(0, A)]

dist 배열

A	B	C	D	E
0	∞	∞	∞	∞

우선순위 큐에는 거리와 노드 enqueue

dist 배열에는 출발점에서 해당 노드까지의 최소 거리 저장

Relaxation

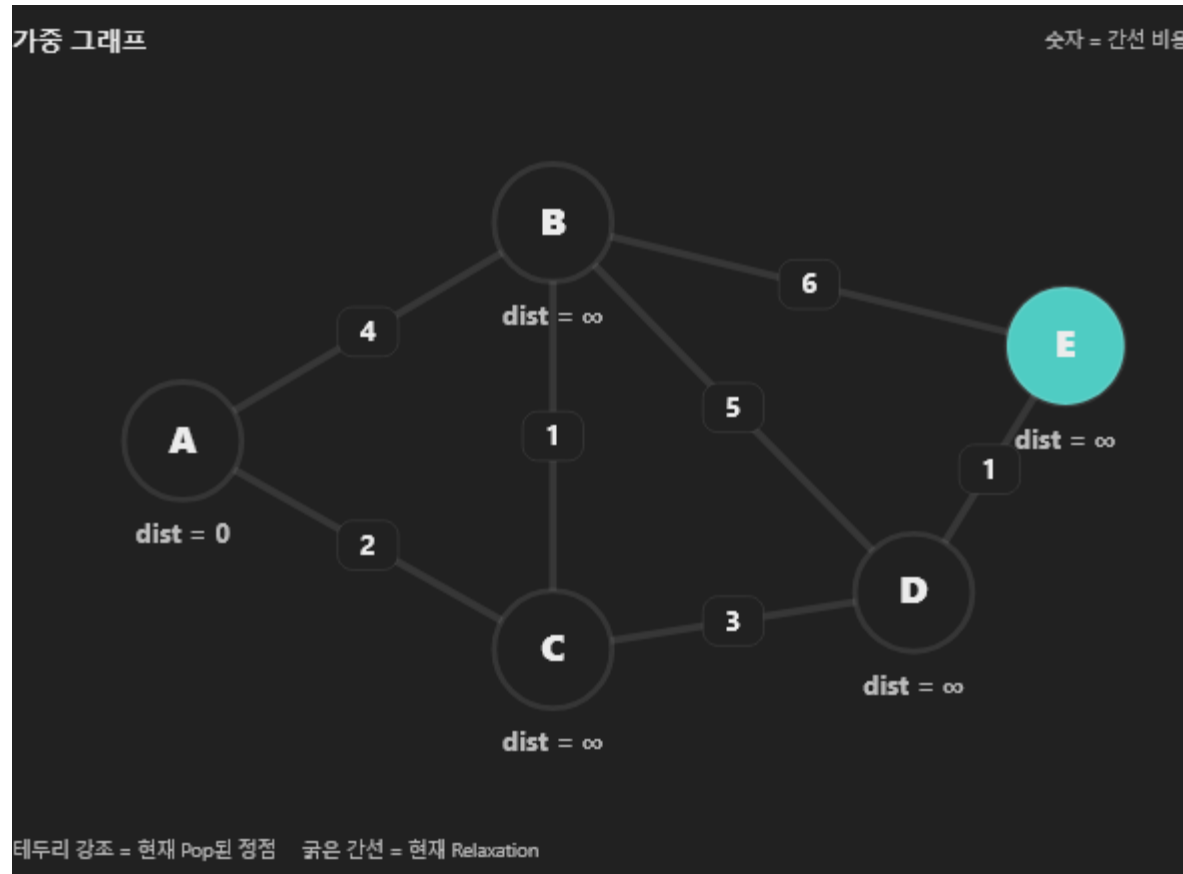


계산 방법

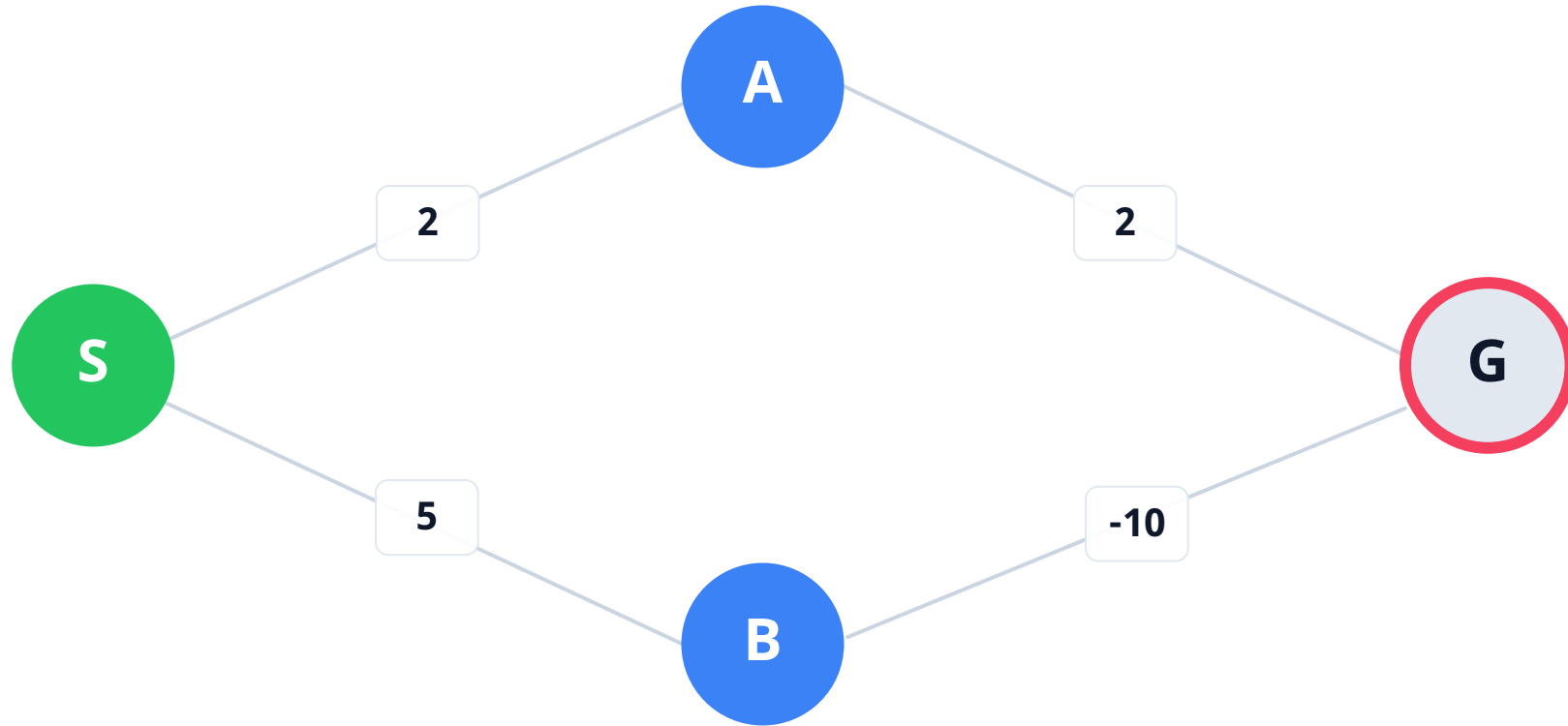
1. 기존 $\text{dist}[v] = 10$
2. $\text{new_dist} = 4 + 3 = 7$
3. $7 < 10$
4. $\text{dist}[v] = 7$

탐색 중 더 짧은 거리를 발견하면
거리를 줄여서 저장

Dijkstra Algorithm

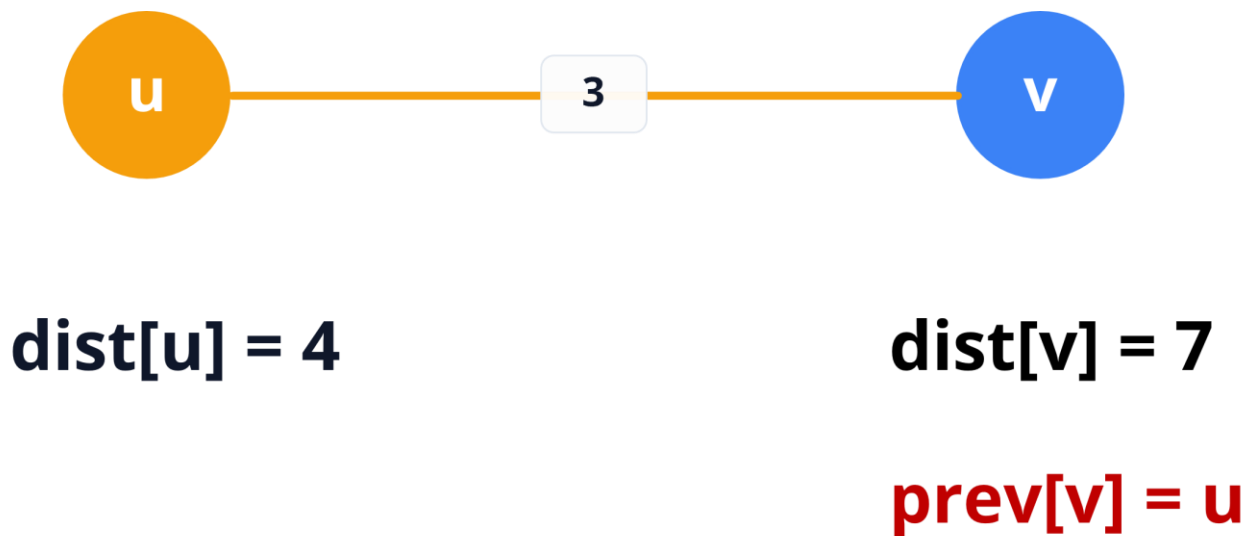


음수 간선일때의 문제



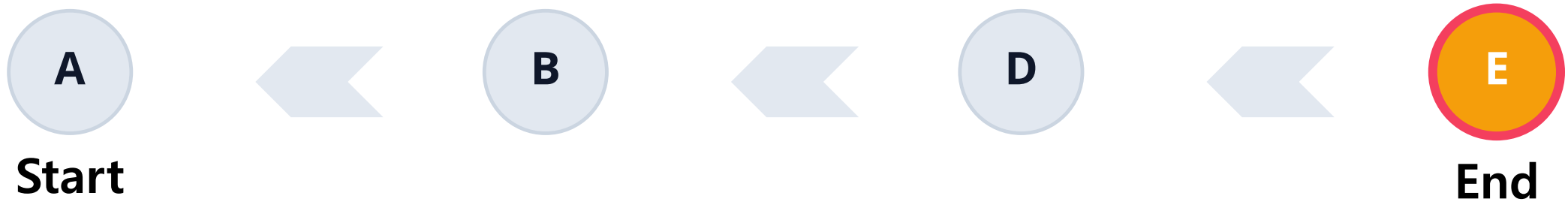
일반적으로 음수 간선이 존재하면 다익스트라를 쓸 수 없음

목적지를 찾아가는 방법



Relaxation 성공 시 이전 노드 저장

Path Reconstruction



E → **D** → **B** → **A** reverse() **A** → **B** → **D** → **E**

우선순위 큐가 비었으면 (다익스트라가 끝났으면)
prev 배열을 뒤집어서 목적지까지의 경로 재구축
(Path Reconstruction)

Weighted Grid

- 시작(S)부터 도착(G)까지 길을 찾아야 함
- 어떤 셀에 방문하면 그 셀에 적힌 숫자만큼 총 거리가 증가한다 가정
- 일반적인 DFS로는 최단거리를 구할 수 없음

S	1	5	1
2	8	2	1
1	1	3	G

Grid Dijkstra

S	.	.	.
.	#	.	.
.	.	.	G

BFS

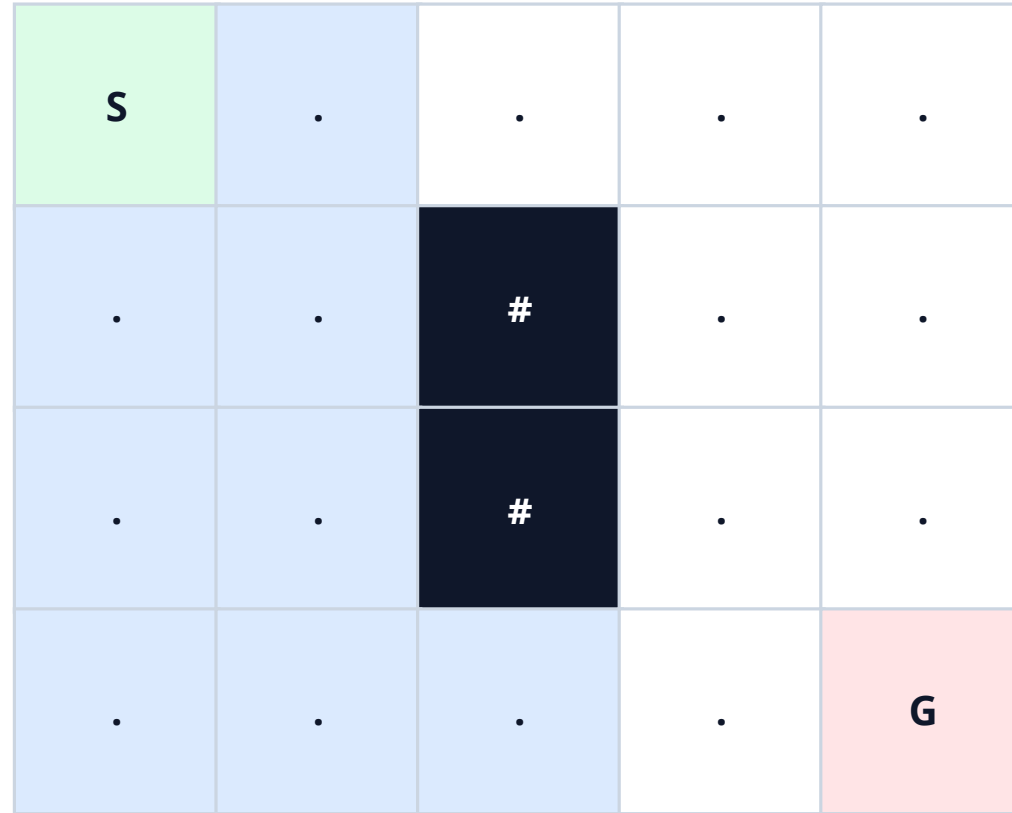
Queue · Level Wave
모든 가중치가 같거나
가중치 자체가 없을 때 사용 가능

S	1	5	1
2	#	2	1
1	1	3	G

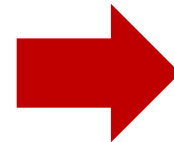
Dijkstra

Priority Queue · Cost Wave

Grid Dijkstra



만약 “목적지까지 얼마나 남았는가”
에 대한 정보를 사용한다면?



A*

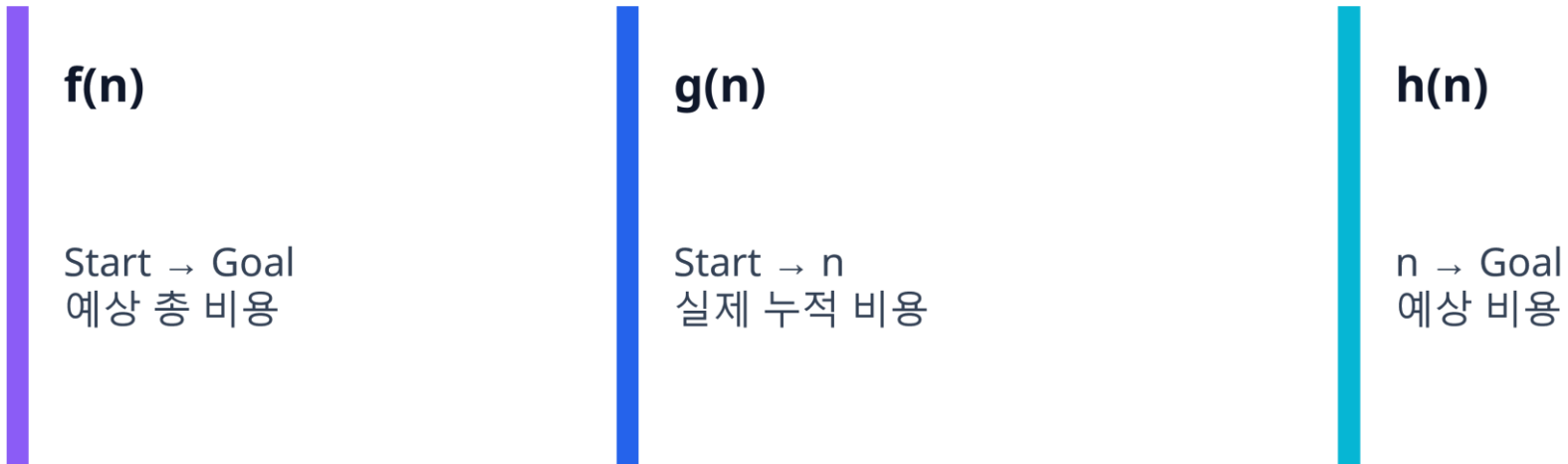
03

A*

A* Algorithm

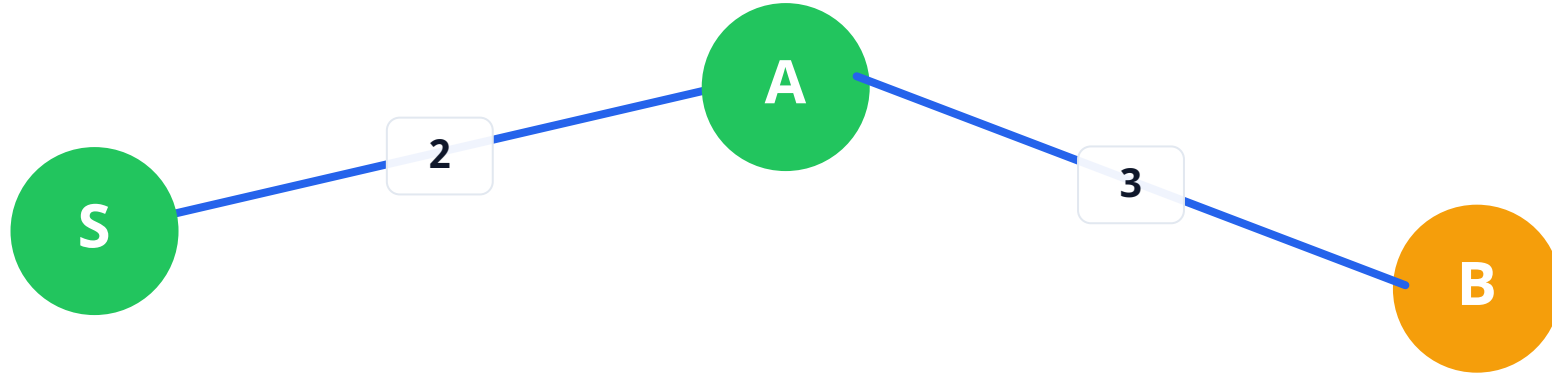
A*

$$f(n) = g(n) + h(n)$$



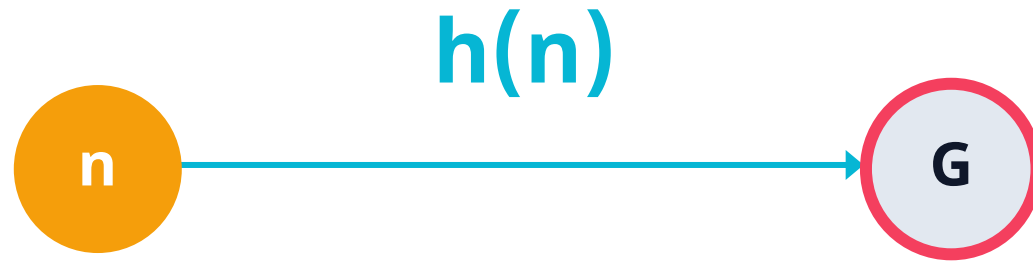
A* 는 이 3개의 값으로 구성됨

$g(n)$: 출발점에서 현재 노드까지의 거리



$$g(B) = 2 + 3 = 5$$

$h(n)$: 현재 노드에서 도착점까지 남은 거리의 추정치

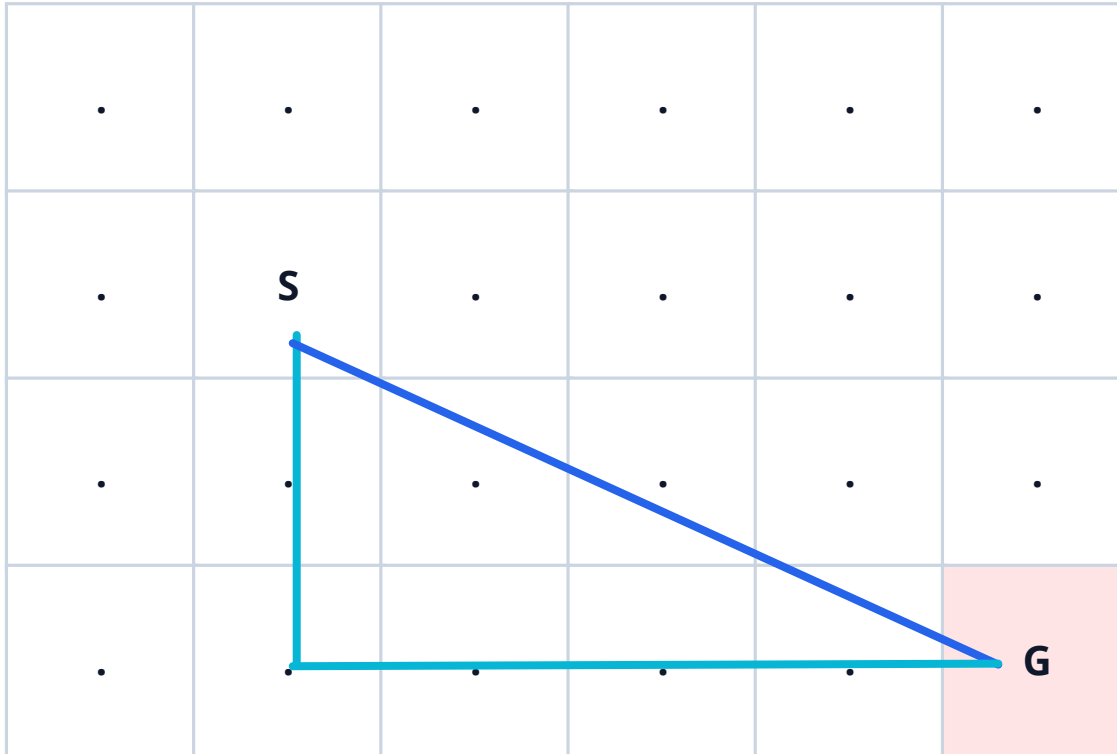


목적지까지 남은 비용을 대충 계산함

이를 Heuristic(휴리스틱)이라 한다

보통 그리드에서는 맨해튼 거리를 주로 씀

Manhattan Distance



Manhattan Distance

6

Euclidean Distance

$2\sqrt{5}$

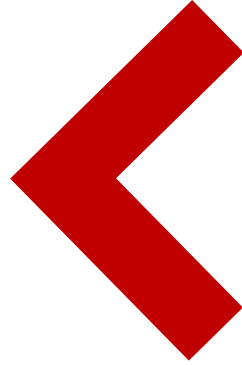
가로세로로만 움직였을 때의 거리

$f(n)$: A*가 비교하는 우선순위

후보 A

$$\begin{aligned}g &= 5 \\h &= 7\end{aligned}$$

$$f = 12$$

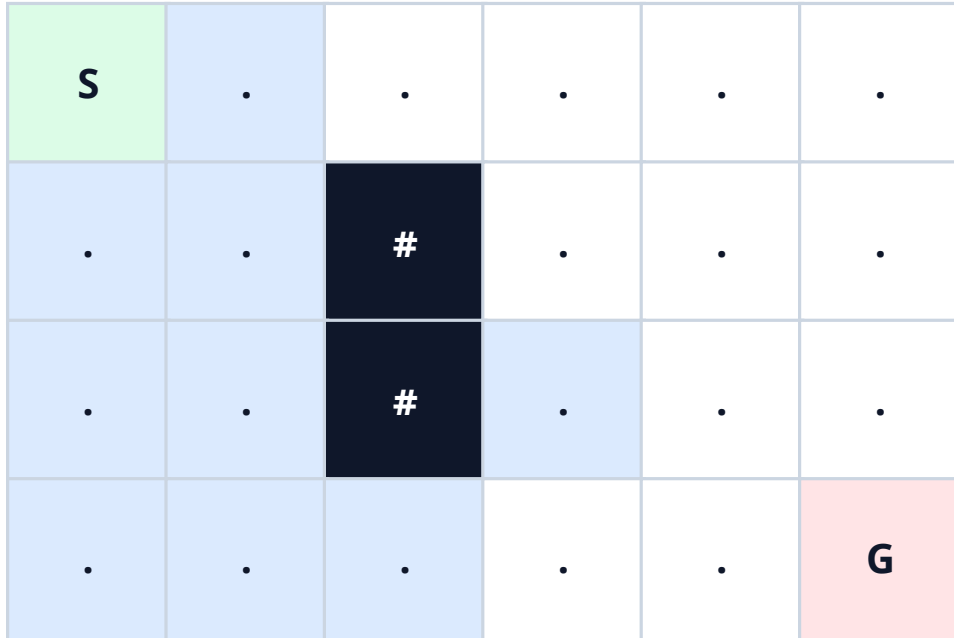


후보 B

$$\begin{aligned}g &= 8 \\h &= 2\end{aligned}$$

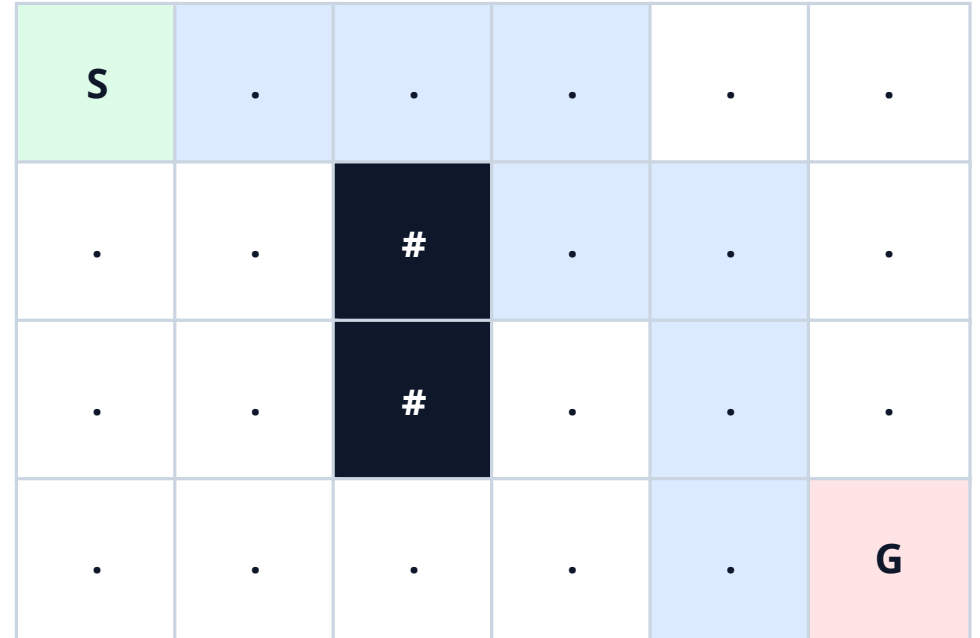
$$f = 10$$

Dijkstra & A*



Dijkstra

priority = g



A*

priority = g + h
(h(n) = 0이면 다익스트라와 동일)